Análisis Arquitectónico y Matemático de MambaVision: Implementación en PyTorch y Migración a Keras 3

Introducción al Paradigma Híbrido Mamba-Transformer en Visión por Computadora

La evolución de las arquitecturas de representación visual ha estado históricamente dominada por una dicotomía fundamental entre el sesgo inductivo local y la capacidad de modelado del contexto global. Las Redes Neuronales Convolucionales (CNN) destacan en la extracción de características jerárquicas locales mediante campos receptivos definidos, pero sufren limitaciones intrínsecas para capturar dependencias espaciales de largo alcance sin recurrir a redes excesivamente profundas. Por el contrario, los Vision Transformers (ViT) introdujeron el mecanismo de autoatención (self-attention), permitiendo un enrutamiento de información global y dinámico desde las primeras capas. Sin embargo, este mecanismo impone un cuello de botella computacional severo: su complejidad de memoria y tiempo es cuadrática con respecto a la longitud de la secuencia de entrada, es decir, $\mathcal{O}(N^2)$, lo que prohíbe pragmáticamente su uso nativo en imágenes de muy alta resolución o en tareas de predicción densa sin parches de gran tamaño.[1]

Recientemente, los Modelos de Espacio de Estados (SSM, por sus siglas en inglés), y más específicamente la arquitectura Mamba basada en el mecanismo de Selección Dinámica (S6), han irrumpido como una solución matemática elegante a esta dicotomía. Mamba logra una complejidad computacional estrictamente lineal $\mathcal{O}(N)$ respecto a la longitud de la secuencia, manteniendo simultáneamente la capacidad teórica de modelar un contexto de longitud infinita mediante un estado oculto recurrente paralelizable.[1] No obstante, la traslación directa de los modelos Mamba desde el dominio del procesamiento de lenguaje natural (NLP) hacia la visión por computadora ha revelado deficiencias empíricas. La naturaleza inherentemente causal y unidireccional de los SSM originales restringe la comprensión espacial bidireccional y multidimensional que requieren las imágenes. Además, las representaciones markovianas profundas tienden a exhibir una degradación o "olvido" de detalles espaciales de alta frecuencia a medida que la secuencia se procesa a través de múltiples capas.[3]

En respuesta a estas limitaciones fundamentales, emerge MambaVision, un backbone de visión híbrido de vanguardia meticulosamente diseñado para reconciliar la eficiencia lineal de los SSM con la superioridad contextual de los Transformers y la robustez local de las CNN. Esta arquitectura híbrida introduce un rediseño profundo de la formulación original de Mamba, creando un bloque denominado *MambaVision Mixer* que elimina las convoluciones causales

restrictivas e incorpora una rama simétrica paralela para preservar la fidelidad espacial.[4] Al acoplar estos mezcladores eficientes con bloques de autoatención ubicados exclusivamente en las etapas finales más profundas de la red, MambaVision establece una nueva frontera de Pareto en el estado del arte (SOTA), superando sistemáticamente a arquitecturas del mismo tamaño tanto en precisión Top-1 en ImageNet-1K como en rendimiento (throughput) de inferencia en hardware acelerado.[6]

El presente informe técnico proporciona una disección analítica exhaustiva, capa por capa y tensor por tensor, de la arquitectura MambaVision. Se expone la fundamentación matemática rigurosa subyacente a cada sub-bloque, se analiza la sintaxis y semántica de su implementación oficial en el framework PyTorch, y se formula la estrategia algorítmica y funcional exacta indispensable para replicar estas operaciones matemáticas utilizando la abstracción agnóstica de hardware de TensorFlow/Keras 3.[7]

Fundamentos Matemáticos del Modelado de Espacios de Estados Continuos y Discretos

Para replicar con fidelidad matemática el comportamiento del bloque MambaVisionMixer en cualquier framework computacional, es imperativo dominar la formulación teórica de los Modelos de Espacio de Estados Estructurados (Structured SSMs) que constituyen el núcleo de su motor de inferencia. MambaVision fundamenta su extracción de características secuenciales en la teoría de sistemas dinámicos de control, mapeando variables latentes en un espacio continuo antes de discretizarlas para su ingesta en arquitecturas de aprendizaje profundo.[1]

Ecuaciones Diferenciales de Espacio de Estados Continuo

Un Modelo de Espacio de Estados continuo clásico define un mapeo desde una secuencia de entrada unidimensional dependiente del tiempo $x(t) \in \mathbb{R}$ hacia una secuencia de salida $y(t) \in \mathbb{R}$, mediado a través de un vector de estado oculto latente $h(t) \in \mathbb{R}^M$. Este sistema dinámico lineal invariante en el tiempo se rige por un sistema de ecuaciones diferenciales ordinarias de primer orden [1]:

$$h'(t) = Ah(t) + Bx(t)$$

$$y(t) = Ch(t)$$

En esta formulación matricial, los tensores de parámetros ostentan roles geométricos específicos dentro del espacio de características. La matriz $A \in \mathbb{R}^{M \times M}$ actúa como la matriz de evolución del sistema, dictando cómo el estado oculto decae, oscila o transita infinitesimalmente a lo largo del tiempo continuo, capturando así la memoria del sistema. El

vector de parámetros $B \in \mathbb{R}^{M \times 1}$ representa la matriz de control o proyección de entrada, definiendo la influencia de la señal exógena $x(t)$ sobre la actualización del estado latente. Finalmente, el vector $C \in \mathbb{R}^{1 \times M}$ constituye la matriz de observación, encargada de proyectar la información contenida en el estado oculto de alta dimensión $h(t)$ hacia el espacio de salida unidimensional esperado.[9]

Discretización mediante Retención de Orden Cero (Zero-Order Hold)

Las arquitecturas de visión operan inherentemente sobre tensores espaciales discretizados (píxeles o parches de embeddings) en lugar de flujos continuos. Por lo tanto, el sistema dinámico diferencial debe someterse a un proceso de discretización matemática rigurosa. Este proceso está gobernado por un parámetro crítico de resolución o paso de tiempo, denotado como el escalar Δ. MambaVision adopta la regla de discretización conocida como Retención de Orden Cero (Zero-Order Hold o ZOH), asumiendo empíricamente que la señal de entrada $x(t)$ se mantiene constante durante cada intervalo de muestreo continuo de longitud Δ.[11]

Bajo el régimen del ZOH, los tensores continuos A y B sufren una transformación analítica exacta hacia sus equivalentes discretos, denotados como $\bar{A}$ y $\bar{B}$. La integración matemática de las ecuaciones diferenciales sobre el intervalo de tiempo Δ rinde las siguientes transformaciones paramétricas [12]:

$$\bar{A} = \exp(\Delta A)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot (\Delta B)$$

$$\bar{C} = C$$

Donde $\exp(\cdot)$ denota la matriz exponencial calculada a través de su expansión en serie de Taylor, e I representa la matriz identidad de dimensión $M \times M$. La aproximación matemática frecuentemente empleada en implementaciones eficientes de Mamba para $\bar{B}$ se simplifica bajo asunciones de un Δ pequeño a $\bar{B} \approx \Delta B$, optimizando severamente el recuento de Operaciones de Punto Flotante (FLOPs) durante el paso hacia adelante. La ecuación recurrente discreta resultante, evaluada en el paso discreto t, se formaliza como [12]:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t$$

$$y_t = \bar{C}h_t$$

Transición hacia la Selección Dinámica (Mecanismo S6)

En iteraciones fundacionales de SSMs como S4, los tensores de discretización $\bar{A}$, $\bar{B}$, y $\bar{C}$ permanecían estáticos e invariantes respecto a los datos de entrada, permitiendo que la ecuación recurrente se desenrollase eficientemente como una convolución global unidimensional aplicando el Teorema de Convolución y la Transformada Rápida de Fourier (FFT).[12] La limitación crítica de esta invariancia radica en la incapacidad del modelo para filtrar adaptativamente información basada en el contenido semántico del parche visual actual.

La arquitectura Mamba, que sirve de cimiento para MambaVision, introduce el mecanismo de Selección Dinámica S6. En este paradigma avanzado, los tensores B, C y el escalar temporal Δ mutan de parámetros estáticos a proyecciones dinámicas dependientes de los datos, siendo funciones explícitas de la entrada x_t en cada paso de tiempo discreto.[12] Esta dependencia de los datos dota al modelo de una capacidad similar a las puertas lógicas (gating mechanisms) encontradas en arquitecturas recurrentes como los LSTMs, otorgándole el poder expresivo para ignorar parches visuales irrelevantes (como el fondo uniforme) y priorizar la memorización de características semánticas cruciales. La penalización de este mecanismo dinámico es la pérdida de la equivalencia convolucional, forzando al sistema a ser computado de manera puramente recurrente. Para superar este obstáculo sin destruir el rendimiento del hardware, Mamba emplea un algoritmo de escaneo asociativo paralelo (Parallel Associative Scan) a nivel de hardware, fusionando las operaciones dentro de la memoria estática de acceso aleatorio (SRAM) de las GPUs modernas para sortear la latencia catastrófica de lectura y escritura en la Memoria de Alto Ancho de Banda (HBM).[15]

Macro-Arquitectura Jerárquica de MambaVision

Antes de descender al nivel de los tensores individuales y su traducción en frameworks, es crucial establecer el contexto del flujo de datos a través de la macro-arquitectura global de MambaVision. El modelo procesa una imagen de entrada $X \in \mathbb{R}^{H \times W \times 3}$ a través de cuatro etapas jerárquicas progresivas que disminuyen gradualmente la resolución espacial mientras aumentan la dimensión del canal C, operando de manera análoga a la pirámide de características de arquitecturas clásicas como ResNet o Swin Transformer.[4]

La inicialización de las características visuales ocurre en el módulo "Stem" (Tallo), donde la imagen original se procesa y proyecta en parches superpuestos. El código oficial en PyTorch

implementa este módulo mediante dos capas consecutivas de convoluciones bidimensionales nn.Conv2d con un tamaño de núcleo de 3×3 y un salto (stride) de 2. Esta operación reduce inmediatamente la dimensionalidad espacial a una resolución de $\frac{H}{4} \times \frac{W}{4}$ con una dimensión de incrustación inicial C_1.[12] Para emular esta extracción semántica inicial en Keras 3, el ingeniero debe instanciar dos capas secuenciales de keras.layers.Conv2D(filters=C_1, kernel_size=3, strides=2, padding='same'), garantizando la congruencia en la reducción del mapa de características.

Las Etapas 1 y 2 de la arquitectura prescinden por completo de mecanismos de espacio de estados o atención, confiando exclusivamente en Bloques Residuales Convolucionales para una extracción de características locales ultrarrápida.[10] Estos bloques emplean la formulación clásica $z = \mathrm{GELU}(\mathrm{BatchNorm}(\mathrm{Conv}_{3\times3}(z)))$ combinada con conexiones de salto $z = \mathrm{BatchNorm}(\mathrm{Conv}_{3\times3}(z)) + z$.[12] El uso deliberado de Normalización por Lotes (Batch Normalization) en PyTorch (nn.BatchNorm2d) en estas etapas tempranas de alta resolución maximiza la eficiencia computacional. En Keras 3, esto se traduce isomórficamente instanciando keras.layers.BatchNormalization(axis=-1) acoplado con keras.layers.Conv2D y keras.activations.gelu, recordando siempre que Keras opera por defecto bajo el formato matricial channels_last (B, H, W, C), a diferencia del paradigma channels_first (B, C, H, W) nativo de PyTorch.

El paradigma híbrido se materializa en las Etapas 3 y 4. Entre cada etapa, un módulo de submuestreo o "downsampler" (típicamente una capa nn.Conv2d con stride de 2) reduce la resolución espacial a la mitad e incrementa la dimensión del canal.[12] Dentro de las etapas profundas, donde la resolución ha caído significativamente a $\frac{H}{16} \times \frac{W}{16}$ y posteriormente a $\frac{H}{32} \times \frac{W}{32}$, MambaVision despliega su disposición bifásica. Considerando un número total de capas N por etapa, la primera mitad exacta de las capas ($\frac{N}{2}$) procesa la información empleando secuencialmente bloques MambaVisionMixer entrelazados con perceptrones multicapa (MLP). Inmediatamente después, la segunda mitad de la etapa ($\frac{N}{2}$ capas restantes) sustituye los mezcladores por bloques convencionales de Autoatención Multi-Cabeza (Multi-Head Self-Attention Transformer Blocks).[6] Este diseño asimétrico obedece a un descubrimiento empírico de los autores: los bloques Mamba sobresalen en el modelado rápido de dependencias bidireccionales de corto y medio alcance, pero exhiben entropía de atención hacia características espaciales periféricas, haciendo que el bloque Transformer posterior actúe como un mecanismo de recalibración global imprescindible para restaurar el contexto espacial de largo alcance.[5]

Análisis Exhaustivo y Traducción del Bloque MambaVision Mixer

El MambaVisionMixer constituye la divergencia intelectual primaria respecto a las arquitecturas Mamba precedentes orientadas al análisis de secuencias unidimensionales (como el texto natural o el audio). El diseño arquitectónico ha sido rediseñado expresamente para procesar tensores bidimensionales aplanados, reconociendo que los píxeles adyacentes visualmente poseen una relación simétrica que desafía la lógica de las convoluciones causales unidireccionales.[6] A continuación se proporciona el desglose matemático, la traza de operaciones en el código fuente de PyTorch, y la formulación correspondiente requerida en Keras 3.

1. Normalización de Entrada y Estabilización Numérica

Antes de iniciar cualquier mezcla de tokens, la secuencia de entrada bidimensional aplanada $X \in \mathbb{R}^{B \times L \times D}$ (donde B es el tamaño del lote, L es la longitud de la secuencia resultante de aplanar el alto por el ancho, y D es la dimensión de incrustación) se somete a un proceso de estabilización estadística. MambaVision sustituye el BatchNorm utilizado en las primeras etapas convolucionales por Layer Normalization (LayerNorm) en estas etapas profundas, calculando la media y la varianza a través de la dimensión de las características D para cada token de forma independiente.[16]

Ecuación Matemática:

$$X_{norm} = \frac{X - \mu_D}{\sqrt{\sigma_D^2 + \epsilon}} \odot \gamma + \beta$$

Implementación en PyTorch:

El código emplea nn.LayerNorm(dim, eps=1e-5) aplicada al inicio de cada bloque como parte de una arquitectura de pre-normalización (Pre-Norm), donde la salida del bloque se sumará residualmente a la entrada original no normalizada.

Equivalente en TensorFlow/Keras 3:

En Keras 3, la equivalencia funcional es exacta. Se debe instanciar y aplicar la capa de normalización de capas, configurando explícitamente el eje de operación para evitar comportamientos anómalos dependiendo de la topología del tensor.

Función en Keras 3: keras.layers.LayerNormalization(axis=-1, epsilon=1e-5).

Framework	Definición del Objeto	Parámetros Semánticos
PyTorch	torch.nn.LayerNorm(normalized_shape=dim)	Calcula momentos sobre la última dimensión por defecto.
Keras 3	keras.layers.LayerNormalization(axis=-1)	axis=-1 garantiza que la normalización ocurra sobre D.

2. Proyección Lineal de Entrada y Bifurcación Asimétrica (Split)

Para enriquecer la representación de características, el tensor normalizado X_{norm} se proyecta primero hacia un espacio dimensional ampliado utilizando una capa lineal densa, y posteriormente se bifurca explícitamente en dos rutas de procesamiento independientes a lo largo de la dimensión de los canales. Esta es la base de la "rama simétrica" paralela introducida en MambaVision.[14]

Ecuación Matemática:

$$X_{proj} = X_{norm} \cdot W_{in} + b_{in}$$

$$x, z = \text{Split}(X_{proj}, \text{dim} = -1)$$

El tensor $x \in \mathbb{R}^{B \times L \times D_{inner}}$ se enrutará hacia el núcleo del motor SSM, mientras que $z \in \mathbb{R}^{B \times L \times D_{inner}}$ viajará a través de la rama residual compensatoria.[18]

Implementación en PyTorch: En las representaciones oficiales del algoritmo, esta expansión suele materializarse mediante una función nn.Linear(dim, dim * expand_ratio * 2) y un subsiguiente operador paramétrico de bifurcación, dictado semánticamente como x, z = torch.chunk(x_proj, 2, dim=-1) o equivalentemente a través de torch.split(x_proj, split_size_or_sections=dim_inner, dim=-1).[18]

Equivalente en TensorFlow/Keras 3:

Keras 3 expone el módulo de operaciones unificadas keras.ops que soporta división de tensores agnóstica de hardware de manera análoga a la API funcional de PyTorch.

Función en Keras 3: keras.layers.Dense(units=dim_inner * 2) seguido por la bifurcación

funcional x, z = keras.ops.split(tensor, 2, axis=-1).

3. Convolución Espacial Simétrica (No Causal) y Activación No Lineal

El avance paradigmático más profundo del MambaVisionMixer frente al bloque clásico de *Vision Mamba (Vim)* es el rechazo explícito a la restricción de convoluciones causales unidimensionales. En el modelado del lenguaje, una convolución causal asegura que el token t solo sea procesado considerando tokens históricos $t-1, t-2, \ldots$ para prevenir la fuga de información futura. En tareas de visión, este paradigma impone una asimetría antinatural y destructiva: un píxel central no debe ignorar su contexto espacial situado a la derecha o por debajo.[6]

MambaVision soluciona esto reemplazando las convoluciones causales por convoluciones de profundidad unidimensional (1D depthwise convolutions) regulares con relleno simétrico (padding simétrico). Tanto el tensor x como el tensor z son procesados a través de estas convoluciones independientes, actuando z_{conv} como un mitigador de pérdida de información de alta frecuencia.[6]

Ecuación Matemática:

$$x_{conv} = \mathrm{SiLU}(\mathrm{Conv1D}_{reg}(x))$$

$$z_{conv} = \mathrm{SiLU}(\mathrm{Conv1D}_{reg}(z))$$

Donde la función de activación SiLU (Sigmoid Linear Unit o Swish) intercala una regularización diferenciable fluida: $\mathrm{SiLU}(\alpha) = \alpha \cdot \sigma(\alpha) = \frac{\alpha}{1+\exp(-\alpha)}$.

Implementación en PyTorch: La clase nn.Conv1d en PyTorch está intrínsecamente diseñada para operar sobre tensores con formato espacial channels_first (B, C, L).[21] Dado que el tensor de entrada x yace en el formato matricial auto-atencional (B, L, C), la implementación de PyTorch debe forzar una transposición del tensor usando el módulo einops o la función nativa permute antes y después de la operación de convolución.[18] Además, la convolución se define como separada en profundidad estableciendo el parámetro de agrupamiento (groups) idéntico a la dimensión del canal. Código PyTorch exacto:

```python
```

```
x_permuted = rearrange(x, 'b l d -> b d l') # Transposición
x_conv = self.conv1d_x(x_permuted)        # nn.Conv1d(dim, dim, kernel_size=3, padding=1, groups=dim)
x_silu = F.silu(x_conv)                   # Activación
x_restored = rearrange(x_silu, 'b d l -> b l d') # Transposición inversa
```

Idéntico paradigma operativo es replicado para la rama z con self.conv1d_z. [18]

Equivalente en TensorFlow/Keras 3: Este es un momento crítico donde la filosofía de diseño orientada a componentes de Keras 3 brilla reduciendo la fricción paramétrica. La clase keras.layers.Conv1D por diseño y asunción asume un formato data_format='channels_last'.[23]

Consecuentemente, los tensores x y z pueden pasar libre y nativamente a través de la capa de convolución sin la sobrecarga computacional o el código espagueti de las transposiciones de ejes (permute), manteniendo al mismo tiempo la exactitud matemática de la convolución discreta.[21]

Operación Requerida	Implementación PyTorch	Implementación Keras 3
Definición de Capa Conv1D	nn.Conv1d(in, out, k=3, padding=1, groups=in)	keras.layers.Conv1D(filters=out, kernel_size=3, padding='same', groups=in)
Manipulación del Tensor	Requiere permute(0, 2, 1) bidireccional	Nativo en channels_last (Ninguna acción requerida)
Función de Activación	torch.nn.functional.silu(x)	keras.activations.silu(x)

4. Generación Dinámica de Parámetros de Discretización (Rama SSM)

Dentro de la rama principal del SSM, el tensor de características convolucionado y activado x_{conv} debe actuar como la semilla generatriz para los parámetros estocásticos que guían el mecanismo de selección S6: la escala de paso temporal Δ, el proyector de entrada paramétrico B, y la proyección de observación escalar C.[12]

Ecuación Matemática:

Para lograr máxima paralelización del hardware, la proyección unificada mapea primero x_{conv}

hacia un espacio hiper-dimensional intermedio:

$$X_{dbl} = X_{conv} \cdot W_{x_proj} \in \mathbb{R}^{B \times L \times (R_\Delta + 2N_{state})}$$

Este hiper-tensor intermedio es subsecuentemente escindido asimétricamente a través de la dimensión de características en tres tensores distintos correspondientes a los parámetros crudos:

$$\Delta_{raw}, B_{raw}, C_{raw} = \text{Split}(X_{dbl},)$$

El vector Δ_{raw} es proyectado linealmente por última vez para expandir su dimensionalidad hacia el tamaño del canal D, seguido por una regularización Softplus para asegurar iterativamente que el intervalo temporal inferido sea estrictamente positivo e invertible analíticamente:

$$\Delta = \text{softplus}(\Delta_{raw} \cdot W_{\Delta_proj})$$

Implementación en PyTorch: El código fuente oficial encapsula estas transformaciones usando capas densas y reorganización tensorial intensiva [18]:

Python

```
x_dbl = self.x_proj(rearrange(x_conv, 'b d l -> (b l) d')) # Proyección unificada
dt, B, C = torch.split(x_dbl, [self.dt_rank, self.d_state, self.d_state], dim=-1) # Split
dt = rearrange(self.dt_proj(dt), '(b l) d -> b d l', l=seqlen) # Proyección de escala Delta
B = rearrange(B, '(b l) dstate -> b dstate l', l=seqlen)
C = rearrange(C, '(b l) dstate -> b dstate l', l=seqlen)
```

Nótese el extensivo uso subyacente de einops.rearrange para preparar topológicamente los tensores para ingestión por el motor C++ CUDA nativo de Mamba, que espera estrictamente que los canales dominen la dimensión final para maximizar la lectura de memoria secuencial del núcleo coalescente.[18] Adicionalmente, el cálculo del parámetro A evolutivo (que a menudo se inicializa matemáticamente y requiere cast de tipos) se define estáticamente mediante A = -torch.exp(self.A_log).[18]

Equivalente en TensorFlow/Keras 3: En la arquitectura Keras 3, logramos la equivalencia matemática sin apelar a abstracciones de bibliotecas externas, aprovechando el espacio unificado de keras.ops y operando sobre la configuración natural de los lotes espaciales.[7] La

división asimétrica en keras.ops.split no toma los tamaños de las particiones discretas subyacentes como PyTorch ([rank, state, state]), sino que exige explícitamente los índices posicionales longitudinales absolutos donde la cuchilla algorítmica dividirá el eje dimensional ([rank, rank + state]).

Concepto Matricial	PyTorch (torch / nn)	Keras 3 (keras.layers / keras.ops)
Proyección Hiper-dimensional	nn.Linear(dim, dt_rank + 2 * d_state)	keras.layers.Dense(units=dt_rank + 2 * d_state)
Bifurcación Asimétrica de X_{dbc}	torch.split(tensor, [r, s, s], dim=-1)	keras.ops.split(tensor, indices_or_sections=[r, r + s], axis=-1)
Proyección Escalar Δ	nn.Linear(dt_rank, dim)	keras.layers.Dense(units=dim)
Parámetro A y Estabilidad de Tipos	A = -torch.exp(self.A_log.float())	A = -keras.ops.exp(keras.ops.cast(self.A_log, 'float32'))

5. Mecanismo de Escaneo Selectivo Paralelo (Selective Scan)

El ápice computacional del bloque Mamba reside en la ejecución del mecanismo de escaneo selectivo iterativo sobre la secuencia espacial. Una vez que los tensores paramétricos dinámicos discretizados han sido resueltos matemáticamente por la fase anterior, dictan la evolución diferencial del estado oculto latente h_t y determinan la topología de la salida observada $y_{ssm,t}$.[6]

Ecuación Matemática Recursiva:

Para cada token espacial evaluado en el tiempo indexado $t \in [1, L]$:

$$h_t = \bar{A}_t \cdot h_{t-1} + \bar{B}_t \cdot x_t$$

$$y_{ssm,t} = \bar{C}_t \cdot h_t + D \cdot x_t$$

Donde $D \in \mathbb{R}^M$ es un vector paramétrico adicional estático de conexión residual entrenable

(Skip Connection) que permea información directamente desde la entrada para amortiguar el desvanecimiento del gradiente en topologías densamente recurrentes.[24]

Implementación en PyTorch: A nivel aplicativo oficial, el código de PyTorch elude deliberadamente el cálculo tensorial explícito en Python puro. Desenrollar esta ecuación en un bucle for vectorial induciría una explosión catastrófica del grafo computacional de autograd, y generaría latencia inaceptable debido a las transferencias cíclicas de lectura y escritura entre la SRAM y la memoria global (HBM) del hardware acelerador gráfico (GPU) para materializar los tensores $\bar{A}$ y $\bar{B}$ discretos.[15] Para soslayar este abismo operativo, los autores invocan una subrutina compilada ad hoc optimizada por el framework Triton / C++: selective_scan_fn(x, dt, A, B, C, D) proporcionada externamente por el ecosistema de la biblioteca mamba_ssm.[12]

Equivalente en TensorFlow/Keras 3 y Compilación XLA: Replicar la firma de esta subrutina matemática en un marco estrictamente Keras 3 (con intención de mantener la agnosia de backends como JAX y TensorFlow) representa el desafío arquitectónico cardinal en la migración del SOTA.[7] Careciendo de un envoltorio puente directo a los kernels Triton de Tri Dao, la solución imperativa y rigurosa exige codificar funcionalmente el cálculo matricial utilizando primitivas tensoriales asociativas, y apoyarse imperativamente en el Motor de Álgebra Lineal Acelerada (Accelerated Linear Algebra - XLA) subyacente, el cual en JAX compilará el proceso mediante el constructor jax.lax.associative_scan para fusionar las operaciones recurrentes homólogamente a Triton.

El flujo algorítmico matemático de emulación dictamina:

1. **Integración Continua a Discreta (ZOH Emulado)**: Empleando la sintaxis tensorial universal de Einstein (einsum) para manejar las contracciones complejas entre la matriz estática de evolución temporal Δ dependiente de los datos.
2. **Operador de Escaneo Secuencial o Asociativo**.

Python

```python
# Abstracción Conceptual para Keras 3 utilizando keras.ops puramente funcionales
import keras
from keras import ops

def custom_keras_selective_scan(x, dt, A, B, C, D):
    # Expectativa paramétrica de dimensiones:
    # x: (Batch, Seq_L, Dim)
    # dt: (Batch, Seq_L, Dim)
    # A: (Dim, N_State)
```

```
    # B: (Batch, Seq_L, N_State)
    # C: (Batch, Seq_L, N_State)
    # D: (Dim,)

    # Fase de Discretización (Zero-Order Hold)
    # Ecuación Discreta A_bar = exp(dt * A)
    # Multiplicación tensorial de Einstein para propagación dimensional
    delta_A = ops.einsum('b l d, d n -> b l d n', dt, A)
    A_bar = ops.exp(delta_A)

    # Ecuación Discreta simplificada (Aproximación Mamba estándar para B)
    # B_bar = dt * B
    B_bar = ops.einsum('b l d, b l n -> b l d n', dt, B)

    # Fase de Escaneo: Se requeriría iteración tensorial.
    # En un backend puro de JAX, este bloque de Python sería substituido por
    # jax.lax.scan o jax.lax.associative_scan para paralelización en árbol
    def scan_recurrent_step(h_prev, current_inputs):
        # Desestructuración funcional de las entradas temporales
        A_bar_t, B_bar_t, x_t, C_t = current_inputs

        # Evolución Recursiva del Estado Oculto
        h_t = A_bar_t * h_prev + B_bar_t * ops.expand_dims(x_t, axis=-1)

        # Computación de la Salida de Observación Proyectada
        y_t = ops.sum(h_t * C_t, axis=-1)
        return h_t, y_t

    # NOTA: En la API de backend específica (ej. JAX dentro de Keras 3):
    # _, y_ssm_raw = jax.lax.scan(scan_recurrent_step, init=h_0, xs=(A_bar, B_bar, x, C))

    # Suponiendo y_ssm_raw materializado:
    # Aplicación residual final de D
    y_ssm_final = y_ssm_raw + (x * D)

    return y_ssm_final
```

Advertencia Instrumental: Si un ingeniero despliega este bucle explícito en hardware de producción sin delegar la resolución computacional a la compilación just-in-time (JIT) del motor subyacente XLA, la degradación sistemática de la métrica de Throughput anulará las ganancias algorítmicas de la linealidad de Mamba.[7]

6. Concatenación de Ramas Simétricas y Proyección Residual de

Salida

Tras la propagación de las características espaciales a través de la tubería compleja del S6, el vector resultante y_{ssm} es linealmente reintegrado junto a la rama compensatoria espacial que fue enrutada inicialmente. La genialidad híbrida del MambaVision Mixer consolida ambas señales semánticas mediante una operación de empalme topológico de los canales.[6]

Ecuación Matemática:

El tensor de salida primigenio del Mamba se une por empalme ortogonal con el vector de la rama activada de forma convolucional:

$$Y_{concat} = \text{Concat}(y_{ssm}, z_{conv})$$

Para salvaguardar la compatibilidad dimensional preestablecida en la interfaz de los macro-bloques sucesivos, el hiper-tensor compuesto experimenta una compresión final a través de una matriz lineal densa para converger hacia su estado paramétrico orgánico:

$$Y_{out} = Y_{concat} \cdot W_{out} + b_{out}$$

Implementación en PyTorch: La recombinación y proyección se cristaliza a nivel de código apelando a la biblioteca einops y operaciones torch contiguas [12]:

Python

```
# Concatena el resultado ssm_x con la rama simétrica z activada a lo largo del canal (dim=1)
hidden_states = rearrange(torch.cat([x_ssm, z], dim=1), 'b d l -> b l d')
# Proyecta linealmente la forma dimensional resultante de dim * 2 de nuevo a dim
return self.out_proj(hidden_states) # nn.Linear(dim * 2, dim)
```

Equivalente en TensorFlow/Keras 3: Con total congruencia estructural, Keras 3 provee acceso axiomático a la concatenación paramétrica, ignorando por completo la necesidad de invocar manipuladores semánticos externos como rearrange. La operación se orquesta en su totalidad empleando directrices espaciales nativas orientadas a la última dimensión.[7]

Elemento Computacional	Sintaxis PyTorch	Sintaxis Operacional Keras 3

Fusión (Concatenación)	torch.cat([x_ssm, z], dim=1)	keras.ops.concatenate([x_ssm, z], axis=-1)
Transformación de Salida	self.out_proj(tensor) (usando nn.Linear(dim*2, dim))	self.out_proj(tensor) (usando keras.layers.Dense(units=dim))

Análisis Exhaustivo y Traducción del Bloque Transformer (ViT)

Si bien los módulos convolucionales tempranos y los Mezcladores Mamba intermedios procesan representaciones semánticas eficientemente con una huella paramétrica sub-cuadrática, los modelos puros adolecen fatalmente del decaimiento de contexto o "olvido algorítmico de largo plazo". A altas densidades de recursividad, la cadena markoviana disuelve las interacciones relacionales críticas de los tensores distantes. El mecanismo híbrido contrarresta explícitamente esto desplegando bloques Transformer completos en la segunda mitad analítica ($\frac{N}{2}$) de las arquitecturas densas en las Etapas 3 y 4.[6] Ya que la dimensionalidad resolutiva se ha degradado en estas etapas (ej. mapas de características a 7×7 con 1024 canales), la computación atencional cuadrática se torna inofensiva e incluso marginal en el volumen total de FLOPs procesados.[1]

Un bloque Transformer canónico en MambaVision se fundamenta en dos topologías iterativas cardinales: el módulo de Autoatención Multi-Cabeza (Multi-Head Self-Attention) guiando la mezcla espacial de los parches, y un Perceptrón Multicapa (MLP) modificado dedicado estrictamente a la integración transversal de los canales de representación semántica.[10]

1. Pre-Normalización y Autoatención Multi-Cabeza (MHSA)

El modelado relacional global confía en la arquitectura isomorfa atencional.[10]

Ecuación Matemática:

Cada parche de características en el vector tensorial mapeado se clona algebraicamente en tres dominios latentes independientes definidos por transformaciones isométricas lineales de Consultas (Queries), Claves (Keys) y Valores (Values):

$$Q = X \cdot W_q, \quad K = X \cdot W_k, \quad V = X \cdot W_v$$

La compatibilidad topológica de relevancia semántica global entre todos y cada uno de los parches se sintetiza calculando el producto escalar covariante entre la matriz temporal densa de consultas y la transposición espacial de la matriz de claves. Un escalar matemático suavizador $\sqrt{d_k}$ modera iterativamente esta varianza espectral antes de ser canalizado por un filtro probabilístico *softmax* multidimensional. Esta calibración probabilística se multiplica matricialmente con los hiper-vectores semánticos reales de los valores:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Múltiples proyecciones de atención paralelas o "cabezas" iteran concéntricamente sobre estos mapas, uniendo sus representaciones divergentes vía concatenación horizontal seguida por otra capa de regresión densa lineal estabilizadora.[10]

Implementación en PyTorch: MambaVision apela comúnmente al empleo de primitivas optimizadas por el C++ subyacente para evadir la huella masiva de la matriz de afinidad atencional atómica iterada.[19]

Python

```
# Abstracción semántica utilizada en los repositorios de la arquitectura
norm_x = self.norm(x) # nn.LayerNorm(dim)
# En una representación funcional detallada, el ecosistema utiliza:
# attn_output, _ = self.mha(norm_x, norm_x, norm_x) (nn.MultiheadAttention)
# O bien, fusionado por FlashAttention empleando F.scaled_dot_product_attention
```

Equivalente en TensorFlow/Keras 3: En la arquitectura Keras 3, la invocación de primitivas de hiper-alta eficiencia de atención se encapsula dentro del motor genérico unificado de abstracciones de capas, proporcionando escalabilidad automática de hardware (como FlashAttention sobre GPUs soportadas) sin intervención manual del desarrollador en topologías algorítmicas.[7]

Parámetro Funcional	Dominio PyTorch	Dominio Keras 3
Instanciación Atencional	nn.MultiheadAttention(embed_dim=dim, num_heads=h)	keras.layers.MultiHeadAttention(num_heads=h, key_dim=dim//h)

Firma Predictiva Computacional	out, _ = self.attn(query=x, key=x, value=x)	out = self.attn(query=x, value=x, key=x)

Precaución de Migración: El vector key_dim exigido estáticamente en la firma semántica del constructor de keras.layers.MultiHeadAttention no representa la dimensión matricial iterada de la entrada monolítica (como lo hace el campo embed_dim de su homólogo en PyTorch), sino rigurosamente el parámetro escalar correspondiente a la dimensión geométrica intrínseca a la cual será proyectada cada cabeza de atención singular de forma algorítmica y asilada.

2. Formulación Paramétrica del Bloque Perceptrón Multicapa (MLP)

Culminando la mezcla geométrica cruzada del sub-bloque de atención iterado, los tensores de características se inyectan en un Perceptrón Multicapa denso operando singularmente por cada parche del conjunto para unificar el modelo semántico del canal transversal. Contrariando fuertemente el axioma estándar establecido en la literatura tradicional de Vision Transformers que expande sistemáticamente la red intermedia (hidden dimension) del MLP multiplicando paramétricamente por un escalar constante de 4, los ingenieros de MambaVision modulan una desviación crítica hacia la eficiencia computacional parametrizando el rango de la proyección a una proporción oculta inferior, tipificada frecuentemente por un factor hiperbólico de expansión intermedio entre 2.0 y 2.5 respecto de su dimensión orginal.[2]

Ecuación Matemática: El flujo algebraico computacional exhibe expansiones matriciales asimétricas interceptadas por interacciones no lineales gaussianas de error de activación (Gaussian Error Linear Unit - GELU) dictadas empíricamente en la iteración arquitectónica oficial [6]:

$$\mathrm{MLP}(X) = \mathrm{Linear}_2\Big(\mathrm{GELU}\big(\mathrm{Linear}_1(X)\big)\Big)$$

Donde matemáticamente, el hiper-proyector asimétrico evalúa el espectro desde $\mathrm{Linear}_1 : \mathbb{R}^D \to \mathbb{R}^{D_{mlp}}$ seguido de la reconstitución regresiva de mapeo desde $\mathrm{Linear}_2 : \mathbb{R}^{D_{mlp}} \to \mathbb{R}^D$, con una asunción condicional dictada algorítmicamente por la expansión topológica $D_{mlp} \approx 2.5 \cdot D$.[2]

Implementación en PyTorch: El ensamblaje funcional modular exige encadenamiento secuencial rígido a través del marco metodológico modular de la biblioteca profunda subyacente [18]:

Python

```
# Inicialización paramétrica densa atada al ecosistema torch
self.fc1 = nn.Linear(dim, int(dim * mlp_ratio)) # mlp_ratio frecuentemente 2.0 o 2.5
self.act = nn.GELU()                    # Implementación paramétrica atómica
self.fc2 = nn.Linear(int(dim * mlp_ratio), dim) # Proyector de reconstitución
self.drop = nn.Dropout(drop_rate)           # Mitigación del hiper-ajuste topológico

# Operativa Computacional (Forward pass iterado)
x = self.fc2(self.drop(self.act(self.fc1(x))))
```

Equivalente en TensorFlow/Keras 3: En completa armonía con las directivas constructivas del sub-ecosistema backend genérico unificado y la optimización lineal algebraica JIT paramétrica delegada [7]:

Nivel de Operativa Lineal	Entidad Computacional PyTorch	Capa Equivalente en Ecosistema Keras 3
Proyección de Expansión (Linear 1)	nn.Linear(dim, hidden_dim)	keras.layers.Dense(units=hidden_dim)
Integración Analítica No-Lineal	nn.GELU()	keras.activations.gelu(x) o keras.layers.Activation('gelu')
Mitigación Topológica Probabilística	nn.Dropout(p)	keras.layers.Dropout(rate=p)
Reconstitución Contractiva (Linear 2)	nn.Linear(hidden_dim, dim)	keras.layers.Dense(units=dim)

En una vertiente paralela arquitectónica presente en variantes derivadas recientes de Mamba empleadas frecuentemente como aproximación al SwiGLU del ecosistema LLaMa en canalizadores asimétricos mixtos (channel mixers), la sustitución matricial atómica GELU diverge a favor del mecanismo *SwiGLU* paramétrico accionado por compuertas lineales.[2] En el supuesto eventual de que la formulación migrada demande emulación rígida SwiGLU, el ingeniero de Keras 3 carece de hiper-primitiva de capa única y requerirá desdoblamiento topológico geométrico en tensor partido y aplicación cruzada paramétrica algorítmica: x, gate = keras.ops.split(tensor, 2, axis=-1) subsiguientemente proyectado analíticamente a la forma de salida compensatoria keras.ops.silu(gate) * x.[2]

Abstracción Global del Modelo Híbrido en Keras 3 y Conclusión Arquitectónica

El marco constructivo paradigmático adoptado por MambaVision representa, indiscutiblemente empíricamente, una optimización heurística severa contra el decaimiento informacional inherente de sistemas puramente markovianos unidimensionales que batallaban por generalizar exitosamente la multidimensionalidad topológica de parches en alta frecuencia espacial para tareas densas de detección y modelado discriminatorio visual algorítmico global.[1]

La sustitución del paradigma unidireccional purista causal por la hibridación combinatoria atómica de una rama de profundidad de red convolucional de regularización paramétrica paralela complementada estructuralmente con sub-módulos auto-atencionales interinos recuperadores de la asintótica visual y topológica del campo receptivo global del Transformer [4], demarca inequívocamente la hoja de ruta constructiva obligatoria para la frontera de Pareto del Estado del Arte SOTA inferencial futuro de bajo espectro limitacional por sobrecarga métrica paramétrica o estancamiento de caudal pasante throughput.[1]

Para el rol de ingeniería de sistemas abocada inexcusablemente hacia la traducción geométrica de los modelos PyTorch nativos originales investigativos a infraestructuras inter-operacionales escalables agnósticas desplegables hacia hardware TPU y múltiples iteraciones asintóticas aceleradas integradas del motor JAX/TensorFlow, Keras 3 emerge fundacionalmente isomorfo para el empalme algorítmico sin fricción geométrica ni latencia por permutación.[7] Las desviaciones en la topología tensorial intrínseca channels_last sobre tensores de cuatro dimensiones espaciales y vectores unidimensionales atómicos de recurrencia en secuencia, liberan paramétricamente al desarrollador de iteraciones cruzadas computacionalmente obstructivas.[21]

Sin embargo, el axioma absoluto paramétrico cardinal del núcleo computacional XLA (Accelerated Linear Algebra) frente a la directiva C++ paramétrica original de la operación selective_scan_fn nativa sigue constituyendo una frontera que requiere obligatoriamente abstracción paralela procedimental asociativa.[6] Evadiendo los bucles recursivos atómicos matriciales paralizantes algorítmicamente y consolidando asintóticamente con JIT compresión geométrica escalar cruzada, la emulación algorítmica Keras 3 asienta la hibridación matemática Mamba-Transformer lista unánimemente para propósitos de modelaje escalado multi-modal masivo en la literatura SOTA moderna de visión global.

Obras citadas

1. MambaVision: Hybrid Vision Backbone - Emergent Mind, fecha de acceso: febrero 26, 2026, https://www.emergentmind.com/topics/mambavision
2. Adventurer: Optimizing Vision Mamba Architecture Designs for Efficiency - PMC, fecha de acceso: febrero 26, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12574601/

3. CVPR Poster Mamba-Adaptor: State Space Model Adaptor for Visual Recognition, fecha de acceso: febrero 26, 2026, https://cvpr.thecvf.com/virtual/2025/poster/34117
4. [CVPR 2025] Official PyTorch Implementation of MambaVision: A Hybrid Mamba-Transformer Vision Backbone - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/NVlabs/MambaVision
5. [2407.08083] MambaVision: A Hybrid Mamba-Transformer Vision Backbone - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/abs/2407.08083
6. CVPR Poster MambaVision: A Hybrid Mamba-Transformer Vision Backbone, fecha de acceso: febrero 26, 2026, https://cvpr.thecvf.com/virtual/2025/poster/33047
7. Introducing Keras 3.0, fecha de acceso: febrero 26, 2026, https://keras.io/keras_3/
8. Computer Science Aug 2025 - arXiv.org, fecha de acceso: febrero 26, 2026, https://www.arxiv.org/list/cs/2025-08?skip=10275&show=2000
9. Hybrid MambaVision and Transformer-Based Architecture for 3D Lane Detection - MDPI, fecha de acceso: febrero 26, 2026, https://www.mdpi.com/1424-8220/25/18/5729
10. Hybrid MambaVision and Transformer-Based Architecture for 3D Lane Detection - PMC, fecha de acceso: febrero 26, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12473383/
11. MHS-VIT: Mamba hybrid self-attention vision transformers for traffic image detection, fecha de acceso: febrero 26, 2026, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0325962
12. MambaVision: A Hybrid Mamba-Transformer Vision Backbone - CVF Open Access, fecha de acceso: febrero 26, 2026, https://openaccess.thecvf.com/content/CVPR2025/papers/Hatamizadeh_MambaVision_A_Hybrid_Mamba-Transformer_Vision_Backbone_CVPR_2025_paper.pdf
13. MambaVision: A Hybrid Mamba-Transformer Vision Backbone - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2407.08083v1
14. DenVisCoM: Dense Vision Correspondence Mamba for Efficient and Real-time Optical Flow and Stereo Estimation - arXiv.org, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2602.01724v1
15. Here Comes Mamba: The Selective State Space Model | Towards Data Science, fecha de acceso: febrero 26, 2026, https://towardsdatascience.com/here-comes-mamba-the-selective-state-space-model-435e5d17a451/
16. A Comprehensive Survey of Mamba Architectures for Medical Image Analysis: Classification, Segmentation, Restoration and Beyond - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2410.02362v3
17. Mammo-Mamba: A Hybrid State-Space and Transformer Architecture with Sequential Mixture of Experts for Multi-View Mammography - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2507.17662v1
18. MambaVision: A Hybrid Mamba-Transformer Vision Backbone - arXiv.org, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2407.08083v2
19. Melchior: A Hybrid Mamba-Transformer RNA Basecaller - bioRxiv.org, fecha de acceso: febrero 26, 2026,

https://www.biorxiv.org/content/10.1101/2025.01.11.632456v1.full.pdf
20. A hybird Mamba-Transformer Vision UNet for Medical Image Segmentation - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2408.11289v2
21. A quick journey through Conv1D functions from TensorFlow to PyTorch, passing via SciPy., fecha de acceso: febrero 26, 2026, https://medium.com/@stevechange/a-quick-journey-through-conv1d-functions-from-tensorflow-to-pytorch-passing-via-scipy-bf830b922b3e
22. What are the differences between these 2 implementations in TensorFlow and Pytorch?, fecha de acceso: febrero 26, 2026, https://www.reddit.com/r/deeplearning/comments/17jsphf/what_are_the_differences_between_these_2/
23. Keras/TensorFlow equivalent of PyTorch Conv1d - Stack Overflow, fecha de acceso: febrero 26, 2026, https://stackoverflow.com/questions/64052842/keras-tensorflow-equivalent-of-pytorch-conv1d
24. Flops calculation · Issue #21 · NVlabs/MambaVision - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/NVlabs/MambaVision/issues/21
25. Transformers-源码解析-六十八- - 绝不原创的飞龙- 博客园, fecha de acceso: febrero 26, 2026, https://www.cnblogs.com/apachecn/p/18275405
26. Completing the Circle: Building a MambaTransformer from Scratch - GoPenAI, fecha de acceso: febrero 26, 2026, https://blog.gopenai.com/completing-the-circle-building-a-mambatransformer-from-scratch-f92a02381bc8
27. SAM-Guided Concrete Bridge Damage Segmentation with Mamba–ResNet Hierarchical Fusion Network - MDPI, fecha de acceso: febrero 26, 2026, https://www.mdpi.com/2079-9292/14/8/1497
28. Cotton leaf disease detection model focusing on small targets and comprehensive feature extraction - PMC, fecha de acceso: febrero 26, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12635100/
29. Adventurer: Optimizing Vision Mamba Architecture Designs for Efficiency - Computer Science, fecha de acceso: febrero 26, 2026, https://www.cs.jhu.edu/~alanlab/Pubs25/wang2025adventurer.pdf
30. Development of an Intelligent System for Automatic Medical Report Generation, fecha de acceso: febrero 26, 2026, https://dspace.univ-guelma.dz/jspui/bitstream/123456789/18265/1/F5_8_HAMMOUDA_HIBA%20ERRAHMEN_1751980948.pdf